

Dependency Injection, IoC & Loosely Coupled Design — Quick Revision

1. Why Spring Core?

Java already has classes, objects, interfaces etc. Spring Core is not about web apps — it helps **create, manage, and connect objects** cleanly in big applications.

2. Dependency

A dependency = something a class needs to do its job. E.g. `OrderService` needs `EmailService` to send an email.

3. Tight Coupling (Problem)

Tightly coupled = hard to change (stuck with one fixed option). Loosely coupled = easy to change (any option works). Like needing "a car" vs needing "transportation."

4. Tight Coupling in Code

When `OrderService` does `new EmailService()` directly, it's tightly bound to that one class. If requirement changes (email → SMS), you must edit `OrderService` itself — bad design.

5. What Exactly Is Tight Coupling?

One class directly depends on a specific concrete class, making code hard to change, test, and reuse.

6. First Improvement: Interface

Create a `NotificationService` interface; `EmailService` & `SmsService` implement it. Now `OrderService` depends on the interface type — better, but not fully solved.

7. Interface Alone Isn't Enough

Even with interface, `new EmailService()` is still written inside `OrderService` — it's still deciding/creating the implementation. Real problem: **object creation happening in the wrong place** (breaks Single Responsibility & Open-Close Principles).

8. The Solution: Dependency Injection

Instead of creating the dependency inside the class, **provide it from outside** (e.g. via constructor). The class just says "give me something that fits this interface."

9. Object Creation Happens Outside

Now `Main` creates `EmailService` and passes it into `OrderService`'s constructor.

`OrderService` no longer creates or decides anything — it just receives.

10. What Is Dependency Injection?

DI = a class receives the objects it needs from outside instead of creating them itself. Simple rule: "**Don't create your dependency, receive it.**" DI is a general design principle — Spring didn't invent it, Spring just automates it.

11. Benefits of DI

- **Easy to change implementation** - swap Email→SMS without touching `OrderService`.
- **Easier to test** - use fake/mock implementations for testing.
- **More reusable** - same class works with many different implementations.

12. Types of Dependency Injection

- **Constructor Injection** - dependency passed via constructor (preferred, makes required dependency clear).
- **Setter Injection** - dependency passed via a setter method (good for optional/changeable dependencies).
- **Field Injection** - covered later in Spring.

13. What Is IoC (Inversion of Control)?

Earlier, the class itself controlled object creation. After DI, that control moves **outside** the class. This shift of control is called Inversion of Control.

14. Simple Definition of IoC

Earlier: "class creates what it needs." Now: "class receives what it needs." The normal flow of control is reversed (inverted) — control moves from inside the class to outside.

15. Relationship Between IoC and DI

- **IoC** = the principle/idea (control moves outside).
- **DI** = the technique used to achieve that principle (passing dependencies in). So: IoC is the goal, DI is one way to reach it.

16. Where Does Spring Fit In?

In plain Java, `Main` manually creates and wires all objects — fine for small apps, messy for large ones with hundreds of classes. We need something to automatically create, manage, and connect objects — that's Spring.

17. Spring IoC Container

An external system provided by Spring that automatically **creates objects, manages them, injects dependencies, and connects classes together** — replacing the job `Main` was doing manually.

18. What Does Spring Core Do?

Three core jobs: (1) Create objects, (2) Manage objects, (3) Connect objects together. This is the foundation before learning Spring MVC, Boot, JPA, Security.

19. What Is a Bean?

A **bean** = an object that is created and managed by Spring. Every Spring bean is an object, but not every object is a Spring bean (only if Spring creates/manages it).

20. Manual Java Flow vs Spring Flow

- **Plain Java:** `Main` creates `EmailService`, creates `OrderService`, and manually injects one into the other.
- **Spring:** The **Spring IoC Container** creates the beans and injects dependencies automatically — no manual wiring needed.

21. Final Summary (Golden Points)

- DI = classes don't create their own dependencies; dependencies come from outside.
- Benefits: easier to change, test, reuse; more loosely coupled.
- IoC = control of object creation moves from class to external system.
- DI = one way to achieve IoC.
- Spring automates DI/IoC using the **Spring IoC Container**.
- Spring Core = object creation + management + wiring — the foundation of everything else in Spring.